S2 · API y Docker (4 horas)

Caso transversal: **Online Retail II** → *¿volverá este cliente a comprar en los próximos 90 días?*
Proyecto de trabajo: `04-Proyecto-Base/` · documento de referencia: `docs/api-y-contenedor.md`.

Objetivo

Al terminar, la persona asistente puede:

1. Explicar qué le falta a un modelo guardado en disco para ser un servicio y por qué el **contrato** debe salir de la **firma** del modelo.
2. Servir el modelo con **FastAPI** y distinguir **200, 422, 503 y 500**, incluida la diferencia entre rechazar y avisar.
3. Explicar la diferencia entre *vivo* (`/salud`) y *listo* (`/listo`) y seguir una petición por el registro JSON con su identificador.
4. Construir una **imagen Docker** mínima, ejecutarla como contenedor y comprobar que no corre como root.
5. **Comparar** las predicciones del paquete, el `joblib`, el formato MLflow, la API y el contenedor.
6. Leer el primer **workflow de GitHub Actions** que reproduce el pipeline, pasa las pruebas, construye la imagen y la prueba.

Agenda

Min	Bloque	Material
0–30	Revisión de entorno	`slides/` diap. 3 · `checklist.md`
30–90	Teoría	`slides/` diap. 4–17 · `teoria/README.md`
90–225	Práctica (135 min, pausa de 10)	`slides/` diap. 18 · `practica/guia-laboratorio.md`
225–240	Entrega y cierre	`tarea/README.md` · `slides/` diap. 19–21

Contenido de esta carpeta

- `index.html`: índice de la sesión (presentaciones, documentos y qué falta). Se genera con `00-Planificacion/indices-de-sesion/generar_indices.py`.
- `slides/index.html`: presentación interactiva (21 diapositivas; flechas, **O** índice, **S** notas, **F** pantalla completa). La diapositiva 8 es un laboratorio de «¿200, 422 o 503?» con las respuestas reales de la API. (nueve casos de archivo y el servidor sin modelo)
- `guion.md`: guion del docente minuto a minuto.
- `checklist.md`: comprobaciones del docente y del alumnado.
- `teoria/README.md`: resumen de teoría y glosario.
- `practica/guia-laboratorio.md`: laboratorio paso a paso con presupuesto de tiempo y salidas esperadas.

- `tarea/README.md` y `tarea/rubrica.md`: tarea y rúbrica (20/40/40).
- `evidencias/plantilla-api-y-contenedor.md`: plantilla de la evidencia entregable.

Los casos de la práctica están en `04-Proyecto-Base/tests/datos/` (`cliente_ejemplo.json`, `lote_ejemplo.json` y `casos/01…10`), y una prueba comprueba que cada uno da el código y el mensaje que dicen la guía y la presentación.

Qué está validado y qué no

- **Validado (Linux, entorno `env-ia-mlops`):** las 112 pruebas del proyecto base (108 rápidas y 4 lentas); 24 modificaciones deliberadas del código, todas detectadas por alguna prueba; los diez casos de la práctica y la receta de la tarea (añadir `monto_180d` al servicio: el API avisa con 503, 18 pruebas fallan hasta actualizar el contrato y vuelven a verde).
- **Validado en GitHub Actions (Ubuntu con Docker):** el `Dockerfile` y el workflow `deploy/github-actions/ci.yml`, con el proyecto en la raíz del repositorio y en la subcarpeta `04-Proyecto-Base/` (variable `PROYECTO`). La imagen se construye, el contenedor responde, rechaza un campo fuera del contrato (422) y corre con el usuario `10001`. La probabilidad del cliente de ejemplo es idéntica en local y en el contenedor.
- **No validado:** el **Codespace del curso** (`.devcontainer/devcontainer.json`, escrito sin abrir ningún Codespace; probarlo antes de clase con `docker version`), `docker build` en un equipo propio, y los comandos en **Windows** (PowerShell y CMD) de esta sesión. La tarea con un dataset distinto del caso transversal.
- **Cifras de las diapositivas:** con la **muestra** del curso (23 070 filas). La probabilidad del ejemplo es `0.8278889191932133`; otro sistema operativo puede cambiar los últimos decimales.

Docker en esta sesión

Docker no se instala en el equipo local: se usa en un **Codespace** (bloque 4 de la práctica) y, como respaldo, el workflow de GitHub Actions construye la imagen por quien no tenga Codespace.